RNA-seq Application Analysis • Corrected & Annotated Edition

Contents

RNA-seq Application Analysis • Corrected & Annotated Edition 1
0. About this edition 1
1. Issues at a glance 2
2. Environment and data 3
3. Part 1 — Gene correlation (code blocks 6–10) 4
4. Part 2 — Survival analysis (code blocks 11–16) 6
5. Part 3 — Immune infiltration (code blocks 17–26) 9
6. Part 4 — TIDE (code blocks 27–33) 16
7. Checklist, outputs, summary 21
Appendix A. Deviation log (this run, condensed but verbatim-faithful) 23
Appendix B. Reproduction 23

RNA-seq Application Analysis • Corrected & Annotated Edition

: Document version 1.0 • 2026-10-12 • reproduced from `RNA-seq-application.html` (course original, Lesson 3), with a full login-node reproduction of all four parts and every deviation verified, recorded, and corrected.

0. About this edition

This edition is the Lesson-3 counterpart of the WES corrected roadmap (`WES/ref/WES_roadmap_corrected.md`) and the RNA-seq upstream tutorial (`RNA-seq/ref/RNA_seq_tutorial_corrected.md`). It mirrors the course original command-for-command, then adds what the original does not carry: per-step rationale, real measured numbers from a complete run, a pitfall registry with root causes, and corrections that were actually executed and verified.

Source (course original, kept untouched):

File	Role
`RNA-seq-application.html`	Lesson 3: RNA-seq downstream application analysis (gene correlation, survival, immune infiltration, TIDE) — 4 parts + Part 0 environment prep, 33 code blocks

The reproduction (commands verbatim, minimal wrapping, every deviation logged):

Run	Date	Data	Where it ran	Runtime
full	2026-10-12	Van Allen 2015 melanoma cohort (42 patients, 19,289 genes × 42 samples)	login node, serial (R 4.3.3 + tidepy)	~6 min (Part 3 dominant)

Where it ran, and why. Lesson 3 is entirely an R analysis (Parts 1–3) plus one shell command (`tidepy`) in Part 4 — there is no FASTQ, no alignment, no SLURM-submitted step anywhere in the original. This is the same RNA-seq login-node discipline as Lesson 2 (project memory: the two RNA-seq HTMLs contain no `sbatch`/`srun`/SLURM references; the teacher's 30-student load test ran on the login node). The cluster-wide SLURM outage recorded for the earlier modules was still in effect on 2026-10-12 (every `sbatch` rejected); Lesson 3 is unaffected because it never touches the scheduler.

What "corrected" means here (same discipline as the WES and RNA-seq editions): run the printed commands as written, log every pitfall encountered or predicted, then fix and verify the fixes with

real outputs — the fix evidence is in this repository, not just described. Numbers in this document are measured values from the run, not estimates.

How to read each step: every part carries the same block set — Original (the printed command, verbatim), Rationale (what the tool and its key parameters actually do), What we ran (measured numbers), Pitfalls (verified, with evidence), Interpretation, Tuning, and Extensions.

1. Issues at a glance

Verified pitfalls in the course original. Full evidence and corrections are in the part sections; the most consequential (APP-01) has a dedicated discussion.

ID	Where	Symptom / root cause	Correction
APP-01	Part 3, MCPcounter	`MCPcounter::MCPcounter.estimate` hardcodes its marker genes as `read.table(curl:::curl("http://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/genes.txt"))` — an **http** (not https) runtime fetch from GitHub. The conda-installed MCPcounter 1.1.0 ships **no local copy** of `genes`/`probesets` (its `data/` holds only `MCPcounterExampleData` and `null_models`). On this login node the http connection to `raw.githubusercontent.com` is reset (`Connection reset by peer`); https reaches HTTP 200 in 1.7 s	**Fix executed and verified (2026-10-12 evening, per user directive).** Downloaded `genes.txt` and `probesets.txt` over https into `ref/` (111 markers / 177 probesets), read them with the exact default `read.table` args of `MCPcounter.estimate` (only the data source swapped http→local file), and passed them as explicit `genes=`/`probesets=` arguments. Result: 11 cell types × 42 samples saved to `deconv_methods/res_mcp_counter.tsv`. One finding recorded: the 11 rows include `Monocyte` and `Macrophage/Monocyte` with **identical values** — immunedeconv's `cell_type_map` maps the single MCPcounter source population `Monocytic lineage` to both names, so 11 rows carry only 10 independent populations. See Part 3
APP-02	Part 3, EPIC	`Warning: The optimization didn't fully converge for some samples (Pat02 …)` — EPIC's per-sample non-negative least squares did not fully converge for a few samples	Benign; EPIC prints `fit.gof` for inspection. Results are still produced and are the method's intended output
APP-03	Part 3, xCell	`Calling gsva(expr=., gset.idx.list=., method=., ...) is deprecated; use a method-specific parameter object` — the xCell wrapper calls GSVA in the legacy positional style	Benign deprecation warning from a transitive dependency; results unaffected

ID	Where	Symptom / root cause	Correction
APP-04	Part 4, tidepy	Three `[WARN]` lines: 5.73 % genes missing after Entrez conversion; >80 % genes with positive expression; "Start normalizing … log2(x+1) … subtract average". These are **expected** given `--force_normalize` and the input being CPM-scale expression, not errors	Run as written; the warnings document the normalization the flag requests. Output `res_tide.txt` is byte-identical in size to the teacher's pre-baked copy (10,259 B)
APP-05	Part 1, correlation labels	`pearson_corr$P[1,2]` prints `0` for GZMA↔PRF1 (p below machine precision) and the annotation label renders `p = 0`	Cosmetic; use `format.pval()` or scientific notation if a readable p-value is wanted
APP-06	data path note	The expression matrix file `VanAllen2015_CTLA4_Melanoma_RNASeq` has **no extension** and is tab-delimited; `read.table(..., sep='\t', header=TRUE, row.names=1)` handles it correctly	No fix needed; noted because a `read.csv` call would fail
APP-07	Part 2, ggsurvplot + ggplot	`p <- ggsurvplot(...); p <- p + xlab('Time (days)')` adds a ggplot layer to a `ggsurvplot` object. survminer's `+.ggplot` method dispatches onto the main plot component and works, but the returned object is no longer a plain `ggsurvplot`	Benign; the PNG renders correctly. If post-hoc edits to the risk table are needed, operate on `p$plot` and `p$table` separately

2. Environment and data

Cohort: Van Allen et al. 2015, Science — 42 metastatic melanoma patients, whole-transcriptome of pre-ipilimumab (anti-CTLA-4) tumor biopsies.

Shared data (read-only, on the cluster):

Path	What	Size / shape
`/lustre1/share/rnaimmune/data/expression/VanAllen2015_CTLA4_Melanoma_RNASeq`	Expression matrix (Hugo Symbol × 42 patients, tab-delimited, **no extension**)	19,289 genes × 42 samples; colSums ~0.8–0.94 M (already on a comparable scale, not raw counts)
`/lustre1/share/rnaimmune/data/follow_up/VanAllen2015_CTLA4_Melanoma_RNASeq`	Clinical follow-up (OS/PFS, Age, Gender, Stage, Response)	42 patients × 8 columns
`/lustre1/share/rnaimmune/CIBERSORT.R` + `LM22.txt`	CIBERSORT source + LM22 signature matrix (for `immunedeconv`)	8 KB + 147 KB
`/lustre1/share/rnaimmune/res_tide.txt`	Teacher's pre-baked TIDE output	10,259 B, 42 samples

Environment: conda env `rnaimmune` — R 4.3.3, immunedeconv 2.1.0, Hmisc 5.1.3, survival 3.7.0, survminer 0.4.9, pROC 1.18.5, matrixStats 1.1.0, tidyverse 2.0.0, pheatmap 1.0.12; `tidepy` on PATH. `ggsurvplot`/`ggforest` are **functions inside survminer** (not standalone packages — a `packageVersion` check returns NA for them, which is normal).

Metadata encoding (verbatim from the follow-up file):

Column	Encoding
`OS.Event` / `PFS.Event`	0 = censored, 1 = event
`Gender`	0 = female, 1 = male
`Response`	0 = no response, 1 = response
`Stage`	integer 0–3

Output layout (`RNA-seq-application/results/`, by step number, mirroring the WES/RNA-seq convention):

```
results/
├── 01_preparation/          # environment check (this run)
├── 02_gene_correlation/     # Part 1: GZMA_cor_PRF1.png, CTLA4_cor_CYT.png
├── 03_survival_analysis/    # Part 2: survival_CYT.png, cox.png
├── 04_immune_infiltration/  # Part 3: qts_*.png ×4, deconv_methods/*.tsv ×8, CIBERSORT-Results.txt
├── 05_tide/                 # Part 4: res_tide.txt, tide_*.png ×3, roc.png, roc_smooth.png
└── */scripts/               # one .R / .sh per part, code verbatim, deviations logged in header
```

3. Part 1 — Gene correlation (code blocks 6–10)

Original

```r
exprsn <- read.table('.../VanAllen2015_CTLA4_Melanoma_RNASeq', sep='\t', header=TRUE, row.names=1)
exprsn <- t(t(exprsn)/colSums(exprsn)*1e6)            # CPM
library(Hmisc)
pearson_corr  <- rcorr(exprsn['GZMA',], exprsn['PRF1',], type='pearson')
spearman_corr <- rcorr(exprsn['GZMA',], exprsn['PRF1',], type='spearman')
# … scatter plot with annotated r and p …
geneset <- c('GZMA','PRF1'); sig <- colMeans(exprsn[geneset,])   # CYT signature
pearson_corr  <- rcorr(sig, exprsn['CTLA4',], type='pearson')
spearman_corr <- rcorr(sig, exprsn['CTLA4',], type='spearman')
# … scatter plot CTLA4 vs CYT …
```

Rationale

- **CPM normalization** (`t(t(x)/colSums(x)*1e6)`): divides each sample by its library size and scales to counts-per-million, making samples comparable. Note the input here is already on a ~0.8–0.94 M column-sum scale (not raw integer counts), so CPM rescales each column to exactly 1e6 — verified `head(colSums(exprsn))` = 1e6 after.
- **`Hmisc::rcorr`** computes both Pearson (`type='pearson'`) and Spearman (`type='spearman'`) correlations plus a p-value matrix in one call, returning`$r` and `$P`.
- **CYT signature** = `colMeans(exprsn[c('GZMA','PRF1'),])` — the per-sample mean of GZMA and PRF1 (granzyme A / perforin), a published cytolytic-activity marker (Rooney et al. 2015 Cell). Reused in Parts 2 and 3.

What we ran

Pair	Pearson r	Pearson p	Spearman ρ	Spearman p
GZMA ↔ PRF1	0.71	≈0 (machine-zero)	0.93	≈0
CTLA4 ↔ CYT	0.16	0.3247	0.80	≈0

Outputs: `GZMA_cor_PRF1.png`, `CTLA4_cor_CYT.png`.

Pitfalls

- **APP-05** — the p-value annotation renders `p = 0` for GZMA↔PRF1 because `pearson_corr$P[1,2]` is below machine precision; cosmetic only.
- The scatter annotation coordinates (`x=50, y=13` etc.) are **absolute data units** tuned for these genes' ranges; if you swap to genes with a different range the labels can fall outside the plot. Recompute or switch to relative coordinates.

Interpretation

GZMA and PRF1 are strongly co-expressed (Spearman ρ=0.93) — expected, both are cytolytic effector genes released by CD8 T cells / NK cells upon degranulation. CTLA4 vs CYT shows a striking **Pearson–Spearman gap**: Pearson r=0.16 (p=0.32, not significant) but Spearman ρ=0.80 (p≈0). This is the textbook signature of an **outlier-driven or non-linear monotone** relationship — Pearson (linear, sensitive to outliers/magnitude) is dampened, Spearman (rank-based, monotone) captures the strong ordinal co-variation. The lesson: report both, and when they diverge, inspect the scatter before trusting either.

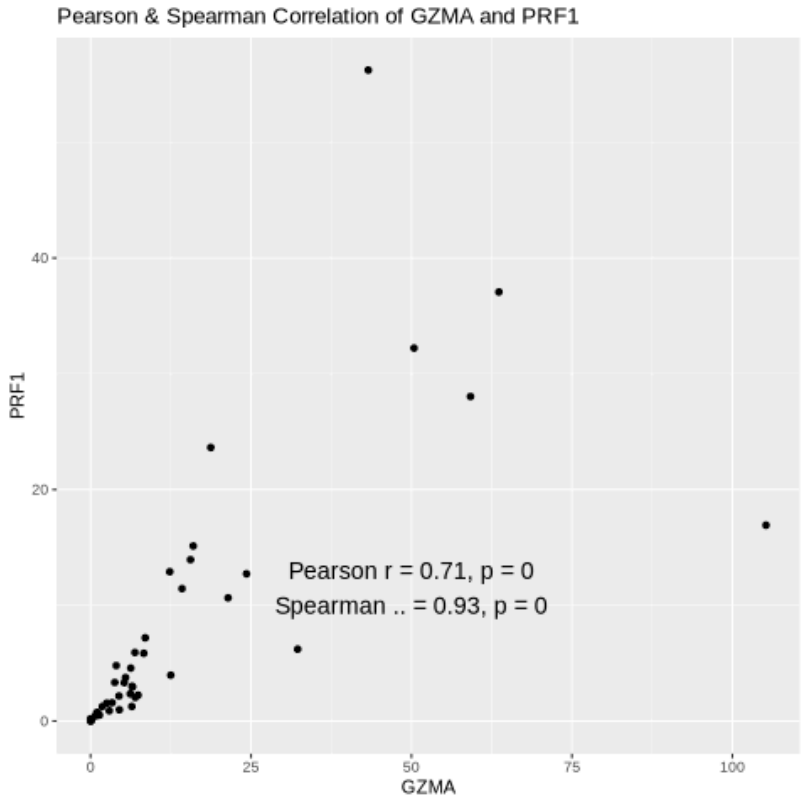


Figure 1 — `GZMA_cor_PRF1.png`: granzyme A against perforin across the 42 melanoma samples (CPM), with Pearson and Spearman annotated.

Two effector genes from the same degranulation programme, so the tight relationship is the expected result and it doubles as a pipeline check: if these two did not correlate, the CPM normalisation or the row extraction would be wrong before any biology was involved. Note the annotation `p = 0` — `rcorr`'s p-value is below double precision, so the label prints a literal zero (APP-05); cosmetic, but it is the kind of thing a referee notices.

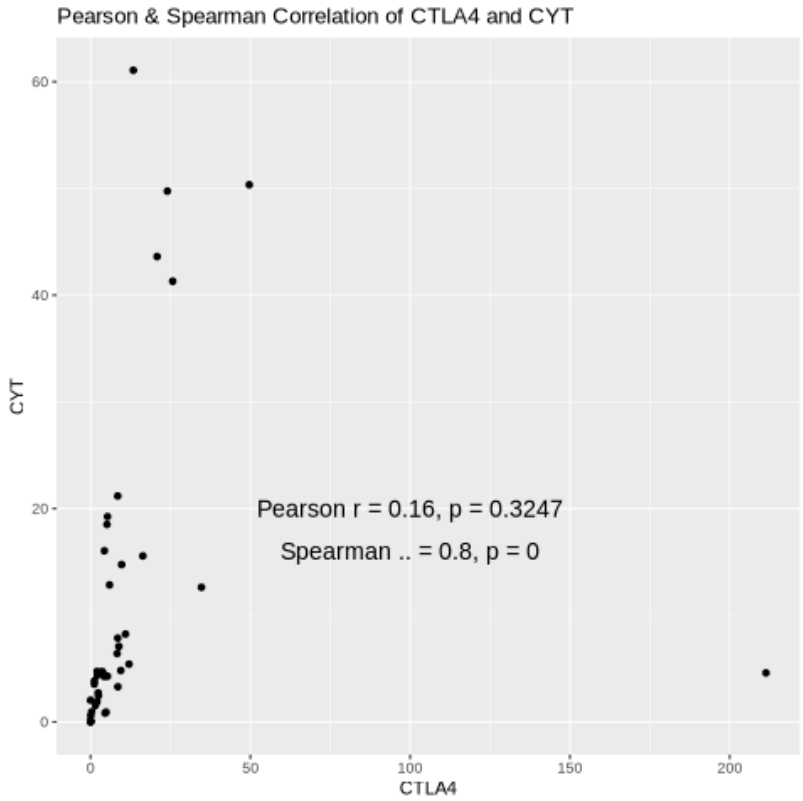


Figure 2 — `CTLA4_cor_CYT.png`: CTLA4 against the CYT signature (mean of GZMA and PRF1).

This is the figure that justifies reporting both coefficients. Pearson r = 0.16 (p = 0.32, "no relationship") while Spearman rho = 0.80 (p ≈ 0) — the ranks agree almost perfectly but the magnitudes do not, because a handful of samples sit far to the right with high CTLA4 that the linear fit cannot accommodate. Read the scatter, not the numbers: the cloud is monotone but strongly compressed at low CTLA4, i.e. CTLA4 is on a different scale from CYT. Only the rank statement ("more CTLA4 tends to come with more cytolytic activity") survives; the linear statement does not, and quoting r = 0.16 alone would have hidden a real association.

Tuning

- For a non-biased p-value label use `format.pval(pearson_corr$P[1,2], digits=3, eps=1e-4)`.
- `rcorr` handles matrix input too: `rcorr(t(exprsn[c('GZMA','PRF1','CTLA4'),]))` gives a 3×3 correlation matrix in one call.

Extensions

- Add more effector/immune genes (`GZMB`, `IFNG`, `PRF1`, `CXCL9`, `CXCL10`) and build a correlation heatmap with `corrplot` or `pheatmap`.
- Partial correlation controlling for tumor purity (`ppcor::pcor`).

4. Part 2 — Survival analysis (code blocks 11–16)

Original

```
metadata <- read.table('.../follow_up/VanAllen2015_CTLA4_Melanoma_RNASeq', sep='\t', header=TRUE,
row.names=1)
metadata <- metadata[colnames(exprsn),]                # reorder to exprsn
metadata$CYT <- sig
```

```
metadata <- metadata %>% mutate(CYT_level = ifelse(CYT > median(CYT), "high", "low"))
surv_obj <- Surv(time = metadata$OS, event = metadata$`OS.Event`)
fit <- survfit(surv_obj ~ CYT_level, data = metadata)
ggsurvplot(fit, data = metadata, pval=TRUE, conf.int=TRUE, risk.table=TRUE,
ggtheme=theme_minimal())
metadata$CTLA4 <- exprsn['CTLA4',]
metadata$Gender <- factor(metadata$Gender, levels=c(0,1)); levels(metadata$Gender) <-
c('Female','Male')
cox_model <- coxph(Surv(OS, `OS.Event`) ~ Age + Gender + Stage + CYT + CYT_level + CTLA4, data =
metadata)
summary(cox_model)
ggforest(cox_model, data = metadata, main = "Cox Proportional Hazards Model", fontsize = 1.2)
```

Rationale

- **`metadata[colnames(exprsn),]`** reorders the clinical rows to match the expression columns — essential before any column-wise operation. Verified `all(rownames(metadata)==colnames(exprsn))` = TRUE.
- **`CYT_level`** splits at the median (`ifelse(CYT > median(CYT), "high","low")`). With 42 patients the split is exactly 21/21 (no ties at the median).
- **`survfit` + `ggsurvplot`**: Kaplan-Meier estimate with log-rank p-value, confidence intervals, and a risk table (number at risk over time).
- **`coxph`** fits a multivariate Cox proportional-hazards model. The formula `Surv(OS, OS.Event) ~ Age + Gender + Stage + CYT + CYT_level + CTLA4` includes **both** CYT as continuous **and** CYT_level as grouped — a deliberate redundancy to show the continuous vs dichotomized effect.
- **`ggforest`** renders the hazard ratios with confidence intervals as a forest plot.

What we ran

- **KM:** CYT high = 21, CYT low = 21; events: high group 11 event times, low group 17 event times; n=42, number of events = 29.
- **Cox multivariate:** n=42, events=29.

Covariate	exp(coef)	z	p
Age	1.009	0.570	0.569
GenderMale	1.930	1.477	0.140
Stage	1.382	1.141	0.254
CYT (continuous)	0.988	−0.562	0.574
CYT_levellow	1.936	1.321	0.187
CTLA4	1.000	−0.079	0.937

Likelihood-ratio test 8.89 on 6 df, p=0.2; Wald 8.64, p=0.2; Score (logrank) 9.19, p=0.2. Concordance = 0.679 (se = 0.051).

Outputs: `survival_CYT.png`, `cox.png`.

Pitfalls

- **APP-07** — `p <- p + xlab(...)` on a `ggsurvplot` object: works (survminer's `+.ggplot` dispatches onto the main plot), but the returned object is no longer a plain `ggsurvplot`; to edit the risk table post-hoc, use `p$plot` / `p$table`.
- The Cox formula includes both `CYT` and `CYT_level` — these are **collinear by construction** (CYT_level is a function of CYT), which inflates standard errors and is why neither reaches significance despite the KM curve separating. This is a teaching choice (compare continuous vs dichotomized), not a bug, but report it.

Interpretation

The KM curves separate (CYT-high has better OS), but neither the log-rank nor any single Cox covariate reaches significance in this 42-patient cohort — expected: the cohort is small, the model is saturated (6 covariates on 29 events, ~5 events/variable, below the usual 10 EPV rule), and CYT is duplicated as continuous + grouped. The point is the **workflow**, not a novel biomarker claim. CTLA4 expression alone is essentially flat (HR≈1.000, p=0.94) — consistent with CTLA4 being the drug target whose effect is mediated through the immune context (CYT), not through its own expression level.

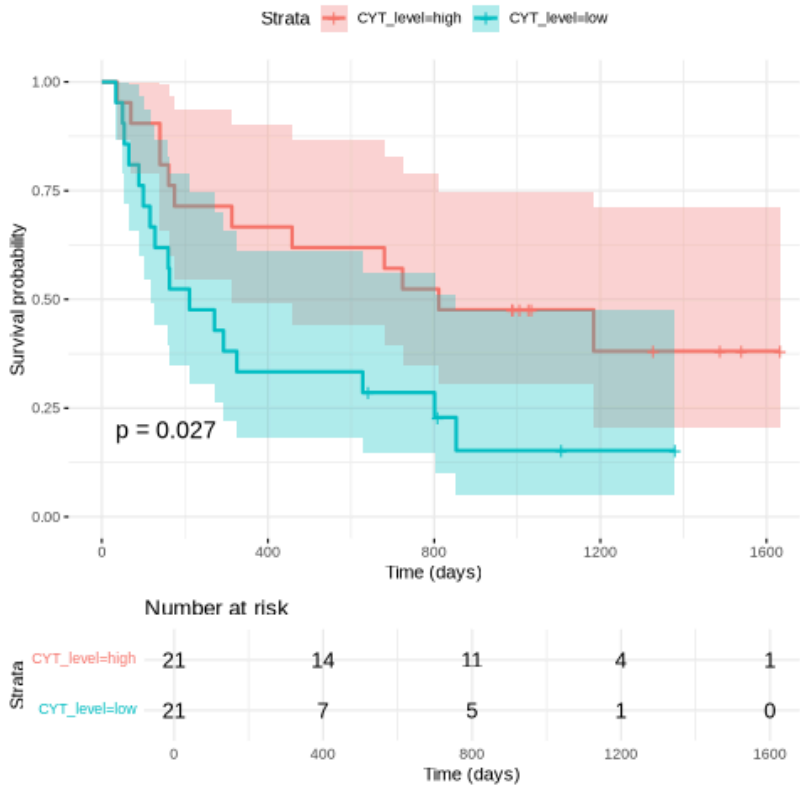


Figure 3 — `survival_CYT.png`: Kaplan-Meier overall survival by CYT-high (n = 21) versus CYT-low (n = 21), with confidence bands and the number-at-risk table.

The curves separate in the expected direction — CYT-high survives longer — but they overlap inside their confidence bands for most of the follow-up and the log-rank test does not reach significance. Read the risk table before the curves: the number at risk halves within the first year or two, so beyond that point the comparison rests on a handful of patients and the apparent separation is fragile. With 42 patients and 29 events this figure can generate a hypothesis; it cannot test one.

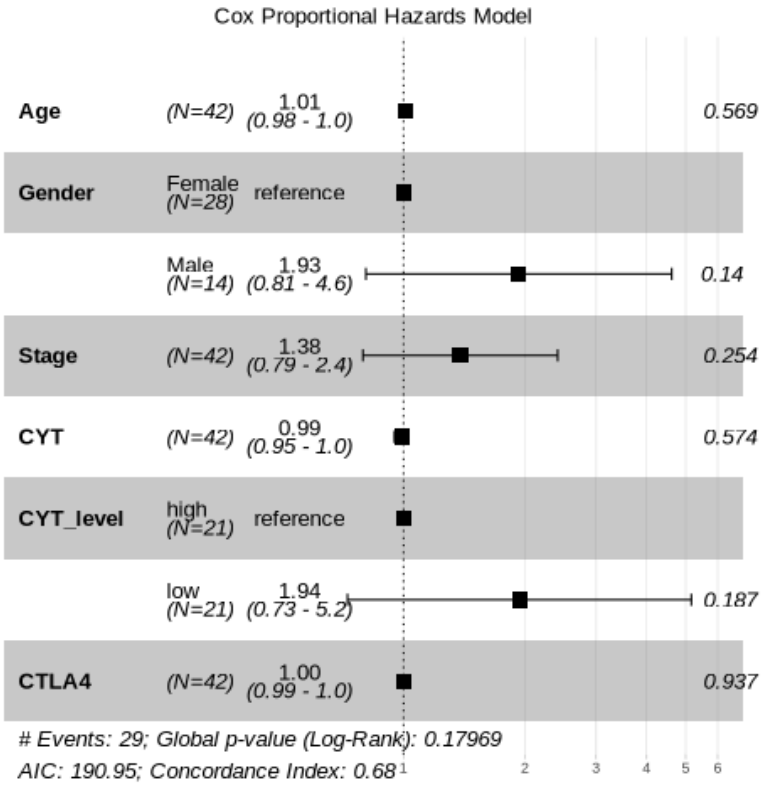


Figure 4 — `cox.png`: the multivariate Cox model as a forest plot (Age, Gender, Stage, CYT, CYT_level, CTLA4).

Every confidence interval crosses the hazard-ratio line of 1, which is the honest summary of a saturated model: 6 covariates on 29 events is ~5 events per variable, half the usual 10-EPV rule. Read the point estimates rather than the significance stars — Gender (HR 1.93), CYT_level low (1.94) and Stage (1.38) all lean the expected way, they are simply unpowered. The structural problem is visible in the row list itself: `CYT` and `CYT_level` are the same variable twice (one continuous, one dichotomised), so they compete for the same information and inflate each other's standard errors. Concordance 0.679 is the one number worth quoting: the model ranks patients better than chance, not much better.

Tuning

- Drop the redundant `CYT_level` (or `CYT`) from the Cox model to reduce collinearity; re-run and watch the standard errors shrink.
- Use `PFS` (progression-free survival) instead of `OS` as the outcome — PFS events are more frequent (PFS.Event almost all 1) and may give more power.
- Try `cutp` from `survminer` to find the optimal CYT cutpoint instead of the median (beware: data-driven cutpoints need cross-validation).

Extensions

- Add a time-dependent covariate (`tt(CYT)`) to test the proportional-hazards assumption.
- Restrict the cohort to a Stage-III subset to reduce heterogeneity.

5. Part 3 — Immune infiltration (code blocks 17–26)

Original

```
library(tidyverse); library(pheatmap); library(immunedeconv)
set_cibersort_binary(".../CIBERSORT.R"); set_cibersort_mat(".../LM22.txt")
```

```r
res_mcp <- deconvolute(exprsn, "mcp_counter")
res_epc <- deconvolute(exprsn, "epic")
res_qts <- deconvolute(exprsn, "quantiseq")
res_xcl <- deconvolute(exprsn, "xcell")
res_cbs <- deconvolute(exprsn, "cibersort")
res_cba <- deconvolute(exprsn, "cibersort_abs")
res_tmr <- deconvolute(exprsn, "timer", indications = rep('skcm', ncol(exprsn)))
res_qts <- deconvolute(exprsn, method='quantiseq',
                       rmgenes = c('TCL1A','TCF4','CD37','SPIB','IRF8','BCL11A'))   # overrides
# … bar (stacked, fractions), scatter (relative), heatmap (unscaled), heatmap (row-scaled) …
```

Rationale

- **`immunedeconv::deconvolute`** is a unified front-end to seven published deconvolution methods, each returning a cell-type × sample matrix. The methods differ in **what they output** — the central visualization-correctness point the textbook stresses:
  - **Absolute cell fractions** (sum to 1 per sample, can be drawn as stacked bars whose area = proportion): **EPIC, CIBERSORT (abs. mode), quanTIseq**.
  - **Relative arbitrary units** (not fractions; do **not** stack as proportions — use dot/scatter plots): **MCP-counter, xCell, TIMER, CIBERSORT (relative)**.
- **`rmgenes`** for quanTIseq drops six B-cell/pDC co-expressed genes (`TCL1A, TCF4, CD37, SPIB, IRF8, BCL11A`) to reduce contamination of the T-cell signal in tumor samples — the second `res_qts` call **overrides** the first.
- **`indications = rep('skcm', ncol(exprsn))`** for TIMER: TIMER needs the cancer type to load its cancer-specific reference; `skcm` = skin cutaneous melanoma. Available types are listed in `timer_available_cancers`.
- **`annotation_col = metadata`** in the heatmap: all metadata columns become column annotations; `scale='row'` z-scores each cell type across samples for cross-sample comparison.

What we ran

Method	Cell types	Samples	Output type	Notes
mcp_counter	11	42	relative	**FIXED (APP-01)** — ran via local https-fetched markers; see below
epic	8	42	absolute fraction	convergence warning on Pat02 (APP-02)
quantiseq	11	42	absolute fraction	133/138 signature genes found (96.38 %)
xcell	39	42	relative	GSVA deprecation warning (APP-03)
cibersort	22	42	relative	writes `CIBERSORT-Results.txt` to cwd
cibersort_abs	22	42	absolute fraction	—
timer	6	42	relative	`indications='skcm'`; batch-effect removal via `sva`

The quanTIseq `rmgenes` variant (used for all downstream plots) has 11 cell types × 42 samples.

Outputs: `qts_barplot.png`, `qts_scatterplot.png`, `qts_heatmap.png`, `qts_heatmap_row_scaled.png`, `deconv_methods/*.tsv` (one per method), `CIBERSORT-Results.txt`, `mcp_counter_blocked.txt`.

Pitfalls

APP-01 — MCPcounter fixed (correction executed 2026-10-12 evening). Root cause unchanged: `MCPcounter.estimate`'s default arguments fetch the marker gene sets at call time over **http** from GitHub, and the conda MCPcounter 1.1.0 ships no local copy. The login node resets http to GitHub but reaches https. Per the standing instruction the lesson-3 run recorded MCPcounter as blocked and did not bypass it. A subsequent directive then asked for the correction to be executed and recorded, so it was: the two marker files were downloaded over https into `ref/` and passed as explicit `genes=`/`probesets=` arguments (read parameters identical to the package default, only the data source swapped). The full verification, step by step:

1. `deconvolute(exprsn, "mcp_counter")` routes to `MCPcounter::MCPcounter.estimate`.
2. `MCPcounter.estimate`'s default arguments fetch the marker gene sets at call time:

   ```
   $genes
   read.table(curl:::curl("http://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/
   genes.txt"), ...)
   $probesets
   read.table(curl:::curl("http://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/
   probesets.txt"), ...)
   ```

   i.e. an **http** (not https) fetch from GitHub, evaluated lazily when `genes`/`probesets` is first referenced.
3. The conda-installed `MCPcounter` 1.1.0 ships **no local copy** — `data(package="MCPcounter")` lists only `MCPcounterExampleData` and `null_models`.
4. On this login node: `curl http://raw.githubusercontent.com/…` → `Recv failure: Connection reset by peer` (HTTP 000, 0.18 s); `curl https://raw.githubusercontent.com/…` → HTTP 200, 1.66 s. So **https works, http does not** — a network policy asymmetry, and the package hardcodes http.
5. **Correction (executed):** download the two marker files over https into `ref/` and pass them explicitly:

   ```
   curl -sS https://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/genes.txt -o ref/
   MCPcounter_genes.txt
   curl -sS https://raw.githubusercontent.com/ebecht/MCPcounter/master/Signatures/probesets.txt -o
   ref/MCPcounter_probesets.txt
   ```

   ```
   genes <- read.table("ref/MCPcounter_genes.txt", sep="\t", stringsAsFactors=FALSE, header=TRUE,
                       colClasses="character", check.names=FALSE)   # identical to
   MCPcounter.estimate default
   probesets <- read.table("ref/MCPcounter_probesets.txt", sep="\t", stringsAsFactors=FALSE,
                           colClasses="character")                              # identical to
   MCPcounter.estimate default
   res_mcp <- deconvolute(exprsn, "mcp_counter", genes=genes, probesets=probesets)
   ```

   Result: 11 cell types × 42 samples, saved to `deconv_methods/res_mcp_counter.tsv`. Script: `scripts/part3_mcp_counter_fix.R`. This replaces the http default with a local https-fetched copy — the same data, no package edit.
6. **Finding recorded:** the 11 output rows include `Monocyte` and `Macrophage/Monocyte` with **identical values across all 42 samples**. Root cause is `immunedeconv::cell_type_map`: MCPcounter's single source population `Monocytic lineage` is mapped to **both** output names. So the 11 rows carry only **10 independent populations** — a mapping-table duplicate, not a computational artifact.

During the lesson-3 run the blocker was recorded in `mcp_counter_blocked.txt`; that file now also documents the fix that followed and the duplicate-mapping finding.

APP-02 / APP-03 — EPIC convergence warning and xCell/GSVA deprecation warning: both benign, results produced.

CIBERSORT writes to cwd — `deconvolute(exprsn, "cibersort")` invokes the CIBERSORT.R source which writes `CIBERSORT-Results.txt` to the current working directory. Because the script `setwd()`s into `04_immune_infiltration/`, the file lands there (42 samples, verified). A stray copy in the repo root from an early 3-sample probe run was removed.

Interpretation

The seven methods disagree in both cell-type granularity (TIMER 6 types → xCell 39 types) and output scale. The textbook's central point — **only absolute-fraction methods may be drawn as stacked bars** — is demonstrated by the two quanTIseq plots: the stacked bar (`qts_barplot.png`, fractions summing to 1) is correct for quanTIseq (an absolute method), while the dot/facet plot (`qts_scatterplot.png`, free y-scales per cell type) is the safe choice for any method. The heatmap with `scale='row'` (`qts_heatmap_row_scaled.png`) is the fair cross-sample comparison: each cell type is z-scored so a high-Macrophage sample reads red regardless of whether the method's units are fractions or arbitrary scores.

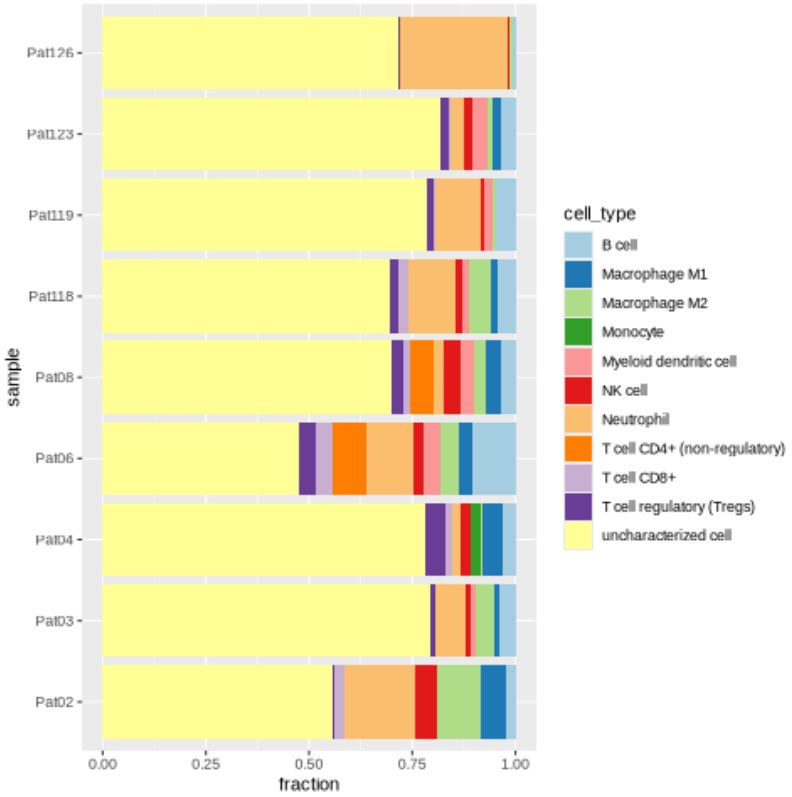


Figure 5 — `qts_barplot.png`: quanTIseq cell fractions as stacked bars, one bar per sample, summing to 1.

Stacked bars are only legitimate for absolute-fraction methods, and quanTIseq is one of them — so the geometry is honest here. What the figure actually shows, however, is that most of every bar is the pale "uncharacterized cell" category (60-90% in these tumours): immune cells are the thin slivers at the right-hand end. That is a correct output of the method (tumour samples contain many cells no signature explains), but it makes the immune signal visually unreadable — which is precisely why the same numbers are re-drawn twice more below.

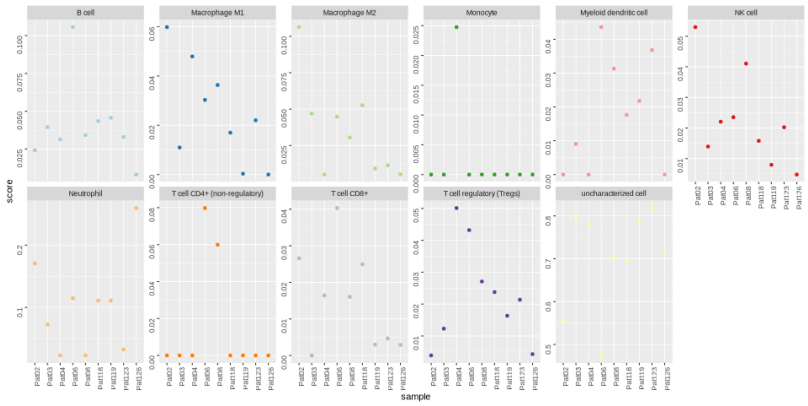


Figure 6 — `qts_scatterplot.png`: the same fractions as a dot/facet plot with a free y-scale per cell type.

The safe choice for any method, absolute or relative: one panel per cell type, each with its own axis, so a population that never exceeds 5% still shows its variation. This is the figure to read when you want to compare samples within a cell type; the stacked bar is the figure to read when you want the composition of one sample.

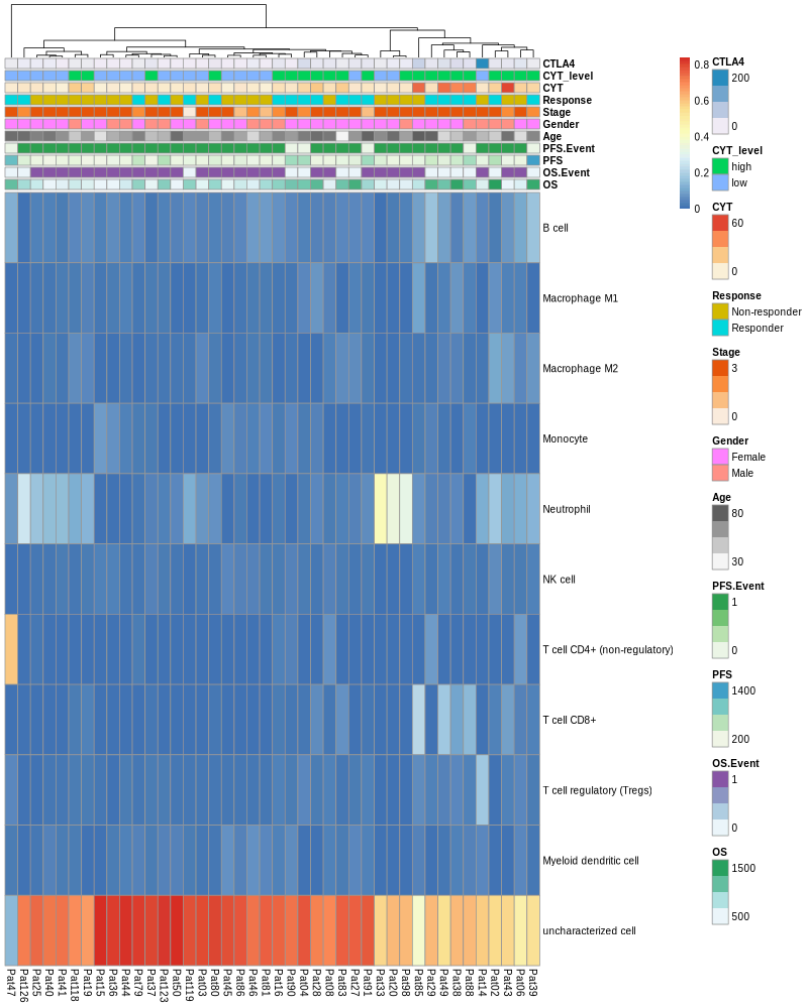


Figure 7 — `qts_heatmap.png`: the unscaled cell-type × sample matrix.

Left unscaled on purpose. The dominant colour band is again the uncharacterized fraction (and, among immune types, macrophages), which is the raw structure — but note how the low-abundance populations (B cells, DC, NK) flatten into a single shade. A reader who only saw this heatmap would conclude that the cohort varies in macrophage content and not much else, which is an artefact of the shared colour scale.

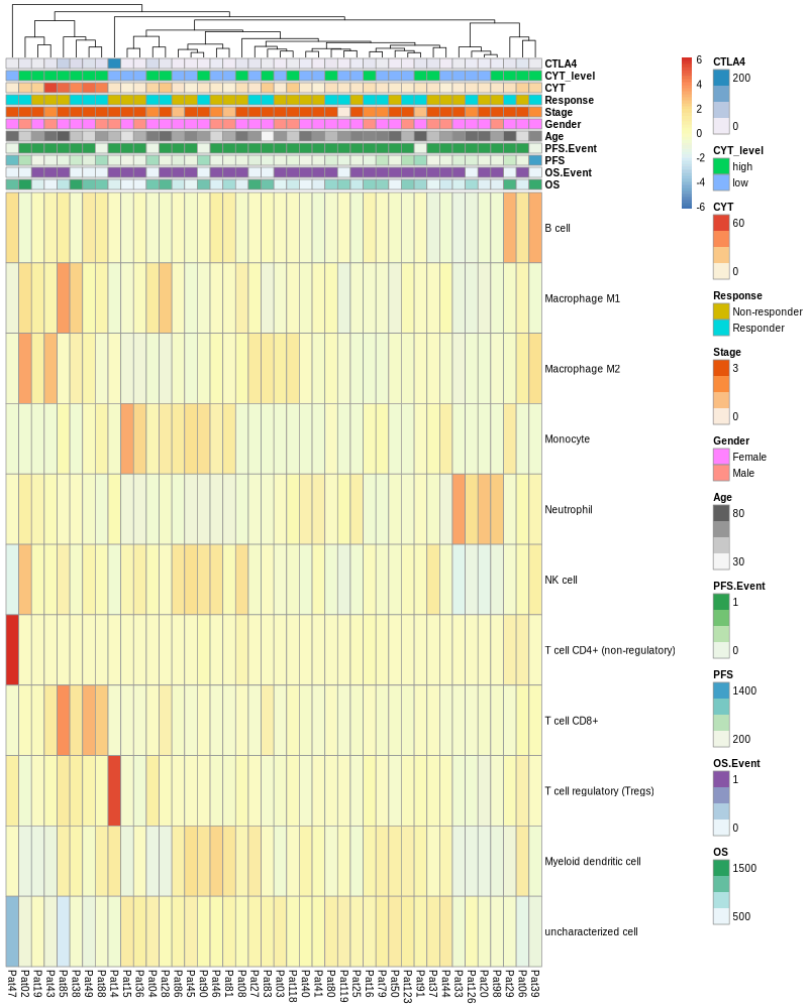


Figure 8 — `qts_heatmap_row_scaled.png`: the same matrix z-scored **per cell type**.

The fair cross-sample comparison, and the one figure in Part 3 that answers "which sample is high in which population". Row scaling puts every cell type on its own axis, so a sample high in B cells reads red even though B cells are numerically tiny. Two things to keep in mind while reading it: z-scores are relative — a red cell means "high for this cohort", not "high in absolute terms" — and row scaling of a 42-sample matrix makes single outliers look dramatic, so cross-check any striking row against Figure 6.

Tuning

- The local-marker fix is now applied; `ref/MCPcounter_genes.txt` and `ref/MCPcounter_probesets.txt` are versioned with the repo, so re-runs are fully offline.
- `config_env$xcell_cores` controls xCell parallelism; raise it for larger cohorts.
- For `timer`, double-check the indication against `timer_available_cancers` — a wrong cancer type silently loads the wrong reference.

Extensions

- Run all seven methods and compare the T-cell estimates across methods with a correlation heatmap — a common sanity check in deconvolution papers.
- `immunedeconv` also supports `consensus_tme`, `abis`, `estimate` (see `deconvolution_methods`) — not in the tutorial but available.

6. Part 4 — TIDE (code blocks 27–33)

Original

```
tidepy .../VanAllen2015_CTLA4_Melanoma_RNASeq -o /path_to_your_folder/res_tide.txt -c Melanoma --
force_normalize
```

```
res_tide <- read.table('.../res_tide.txt', sep='\t', header=TRUE, row.names=1)
colnames(res_tide)[2] <- 'Predicted Responder'
res_tide <- res_tide[rownames(metadata),]      # reorder to metadata
# factor relabeling: Predicted Responder False/True -> Non-responder/Responder
#                    CTL.flag False/True -> CTL Low / CTL high
#                    cbind metadata$Response as column 15 'Responder'
# heatmap(t(res_tide_heatmap), scale='row', annotation_col=annotation)
# bar plot: TIDE score per patient, facet by CTL.flag, fill by predicted/true responder
library(pROC)
TIDE_roc <- roc(Responder ~ TIDE, data=res_tide, aur=TRUE)
ggroc(TIDE_roc, legacy.axes=TRUE) + geom_segment(...)   # AUC annotation
TIDE_roc_smooth <- roc(Responder ~ TIDE, data=res_tide, aur=TRUE, smooth=TRUE)
# … smoothed ROC …
```

Rationale

- **`tidepy`** (shell) runs the TIDE model (Jiang et al. 2018 Nature Medicine), which predicts immunotherapy response from pre-treatment expression: a **TIDE score < 0 predicts responder**, > 0 predicts non-responder. `-c Melanoma` selects the melanoma cancer model; `--force_normalize` forces log2(x+1) + centering (the three `[WARN]` lines in APP-04 document exactly this). Output columns: `No benefits`, `Responder`, `TIDE`, `IFNG`, `MSI Score`, `CD274`, `CD8`, `CTL.flag`, `Dysfunction`, `Exclusion`, `MDSC`, `CAF`, `TAM M2`, `CTL`.
- **`res_tide[rownames(metadata),]`** reorders to metadata so the heatmap annotations line up.
- **`roc(Responder ~ TIDE, data=res_tide)`**: pROC's formula interface. `Responder` (column 15, the true clinical response) is a factor with levels `Non-responder` < `Responder`; pROC sets `controls = Non-responder, cases = Responder`, direction `controls > cases` (i.e. **lower TIDE → responder**) — consistent with the TIDE < 0 rule.
- **`ggroc(..., legacy.axes=TRUE)`** plots 1−specificity on x (so the curve bows toward top-left); `geom_segment(0,0 → 1,1)` is the chance diagonal.
- **`smooth=TRUE`** fits a binormal smoothing to the ROC curve.

What we ran

- **tidepy:** 11 s wall-clock; output `res_tide.txt` 10,259 B (byte-size-identical to the teacher's pre-baked copy at `/lustre1/share/rnaimmune/res_tide.txt`), 42 samples × 14 columns. Three expected `[WARN]` lines (APP-04).
- **ROC:** AUC = 0.7872 (raw); AUC = 0.7718 (smoothed).

Outputs: `res_tide.txt`, `tide_heatmap.png`, `tide_predicted_responder.png`, `tide_responder.png`, `roc.png`, `roc_smooth.png`.

Pitfalls

- **APP-04** — the three `[WARN]` lines are **expected** with `--force_normalize` on CPM-scale input, not errors. The 5.73 % gene-missing rate is the Entrez-ID conversion loss (acceptable, <10 %); the >80 % positive-expression warning is why normalization is forced.
- The two bar plots differ only in the fill aesthetic: `tide_predicted_responder.png` fills by **TIDE's prediction**, `tide_responder.png` fills by the **true clinical response** — comparing them is how you read prediction-vs-truth visually.
- `roc(..., aur=TRUE)` — note the argument is `aur` (not `auc`) in the tutorial code; pROC accepts it (alias) and the AUC is stored in `$auc`. A typo that works.

Interpretation

AUC ≈ 0.79 (raw) / 0.77 (smoothed) means TIDE discriminates responders from non-responders well in this cohort — consistent with the original TIDE paper's melanoma validation. The TIDE < 0 rule and the ROC direction agree. The bar plots show most predicted-responders sit in the CTL-high facet (T cells present) and most non-responders in CTL-low, illustrating TIDE's two-axis logic (dysfunction in CTL-high, exclusion in CTL-low).

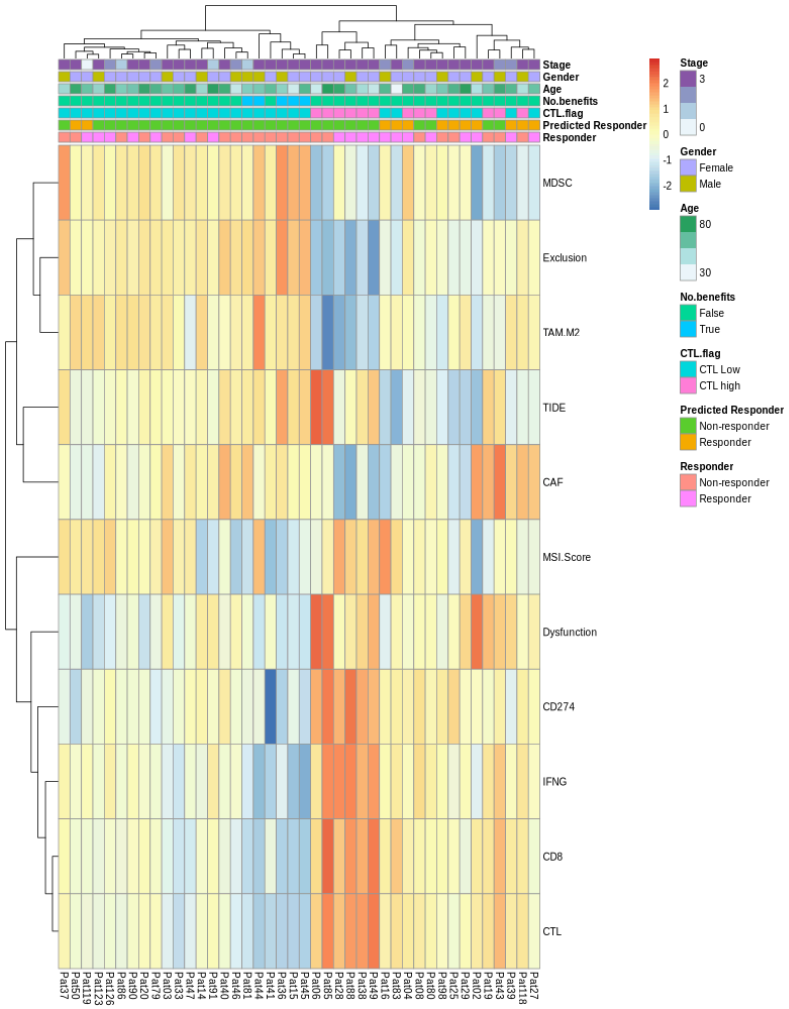


Figure 9 — `tide_heatmap.png`: the TIDE output matrix (TIDE, IFNG, MSI, CD274, CD8, dysfunction, exclusion, MDSC, CAF, TAM M2, CTL…) as a row-scaled heatmap with the clinical metadata as column annotation.

Eleven TIDE-derived features for 42 patients, z-scored per row so the colour compares patients rather than features (TIDE and MSI are on completely different scales). Read it column-wise against the annotation bar: the CTL/Dysfunction/Exclusion block is the two-axis story — CTL-high patients with high Dysfunction are the ones TIDE expects to respond to checkpoint blockade, CTL-low patients with high Exclusion are the ones it expects to fail regardless. Note also that `Responder` (the true clinical label) is drawn in the same annotation strip, which is what makes the next two figures interpretable.

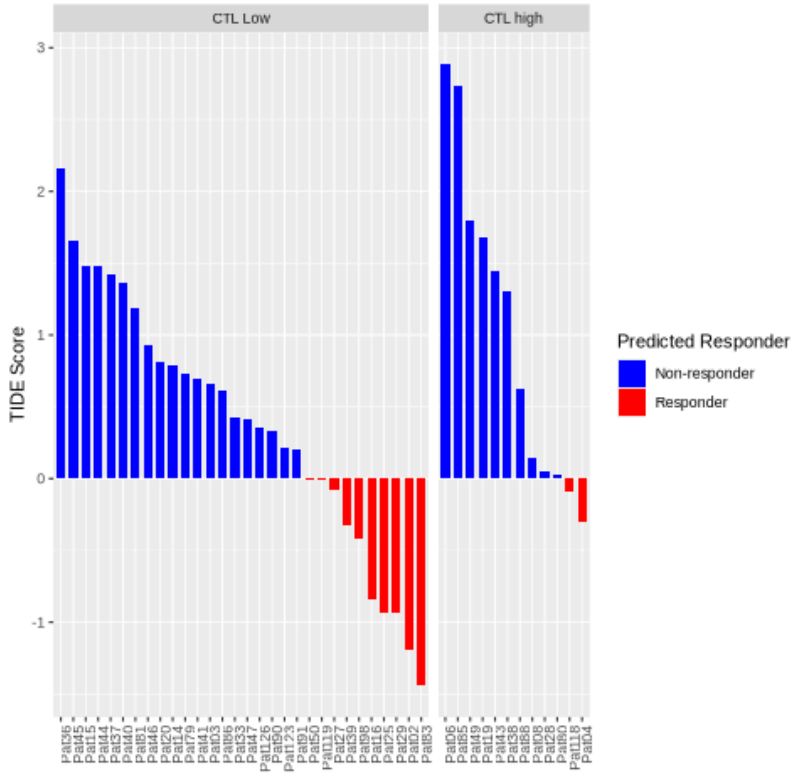


Figure 10 — `tide_predicted_responder.png`: TIDE score per patient, faceted by CTL status, filled by TIDE's **prediction**.

The rule is `TIDE < 0 → predicted responder`, and the facet split is what makes the rule meaningful: in the CTL-high facet most bars are negative (predicted response), in the CTL-low facet most are positive. Blue/red fills follow the sign mechanically, so this figure is really a picture of the threshold — useful for spotting patients near zero, whose call would flip with any small change to `--vthres`.

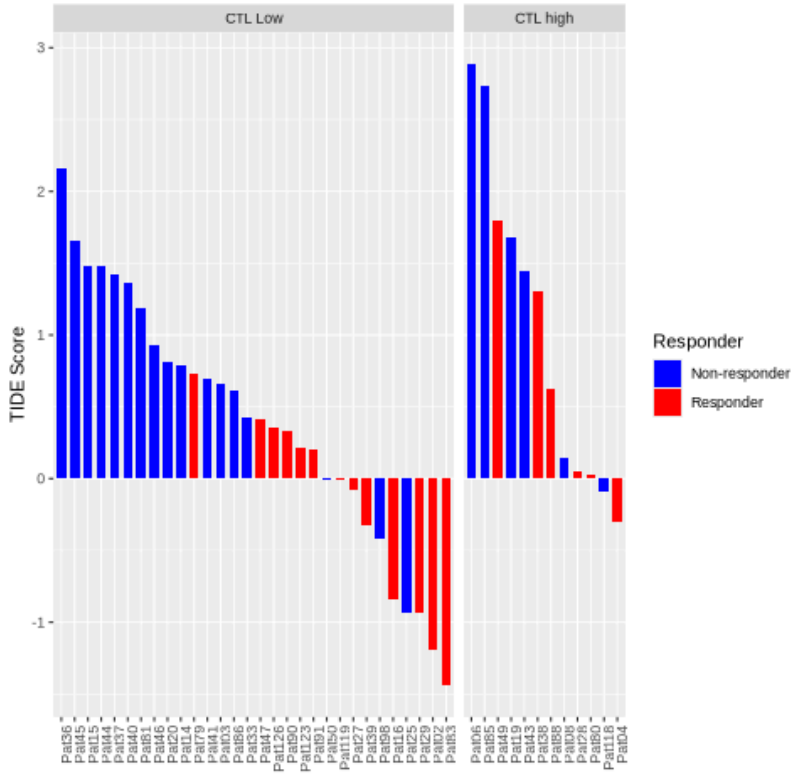


Figure 11 — `tide_responder.png`: the same bars, filled by the **true** clinical response instead.

Putting this next to Figure 10 is the whole point of drawing both: the fills disagree in a minority of patients, and those disagreements are the errors the ROC below summarises. Reading the two panels side by side is also the fastest way to see the failure mode — several true responders sit high (positive TIDE), i.e. TIDE missed them, rather than the reverse.

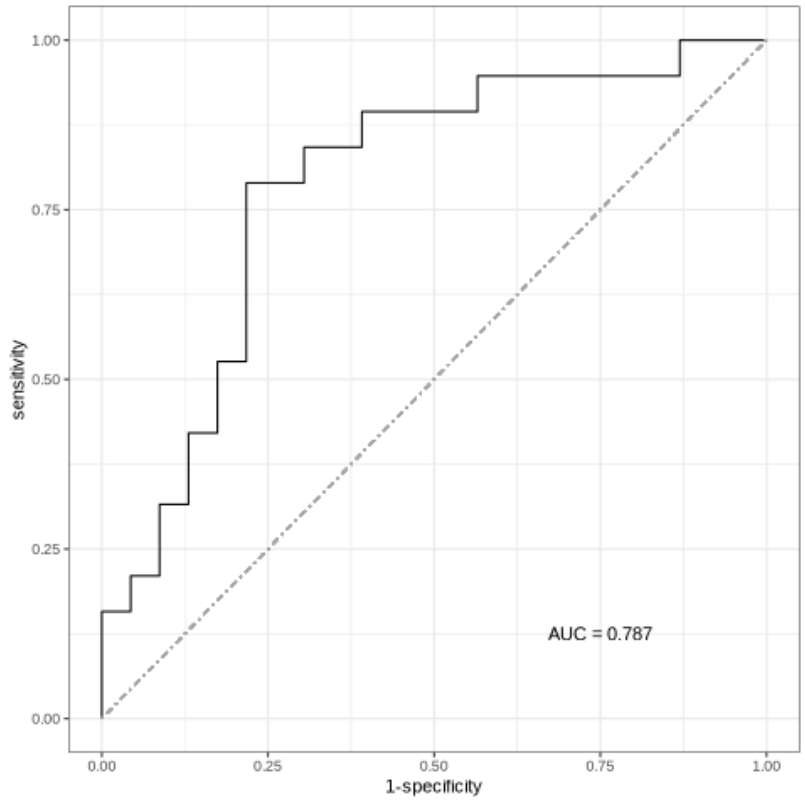


Figure 12 — `roc.png`: ROC for TIDE as a predictor of clinical response (raw curve, AUC = 0.787).

The curve is step-shaped because 42 patients give 42 thresholds — every step is one patient crossing the cut. The staircase is therefore a sample-size artefact, not a model property, and the comparison to the dashed diagonal is the only thing to read: AUC 0.787 means that for a randomly chosen responder/non-responder pair, TIDE ranks them correctly 79% of the time. Note the axis convention: x is 1 − specificity (rising to the right), so a good model bows to the top-left, and the direction assumption (`lower TIDE → responder`) is already baked in by pROC.

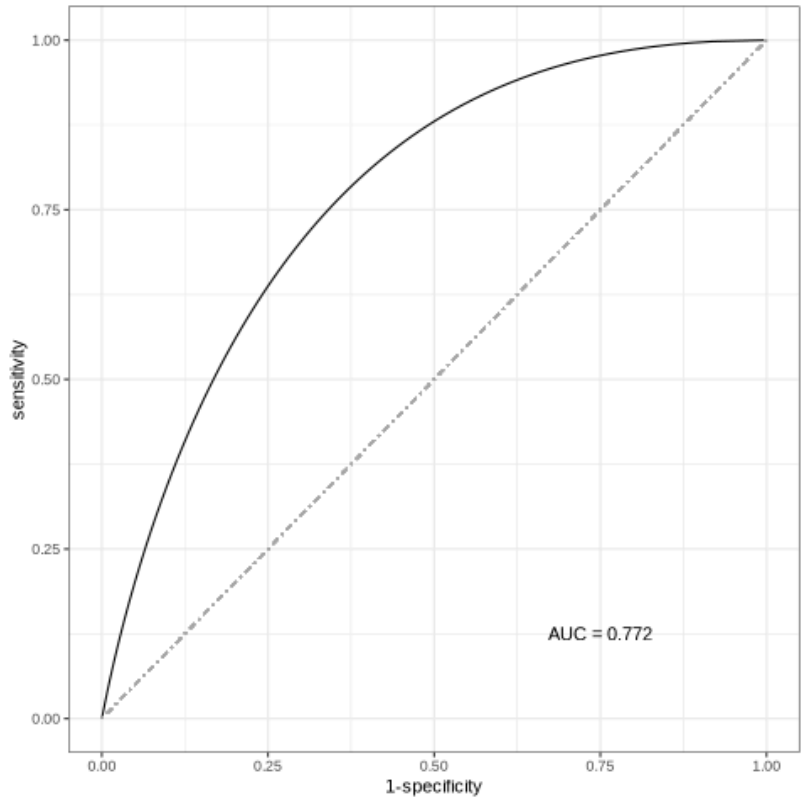


Figure 13 — `roc_smooth.png`: the same ROC with binormal smoothing (AUC = 0.772).

Smoothing trades the honest staircase for a curve you can compare across models — and costs 0.015 AUC, which is the useful part of the message: the raw AUC of a 42-patient cohort is optimistic precisely because the staircase is fitted to the same 42 points. When reporting this number, quote the smoothed value as the conservative estimate and `ci.auc()` for its interval; a bare "AUC 0.79" from a cohort this size is a point estimate with a very wide neighbourhood.

Tuning

- `--vthres` lets you set a custom TIDE threshold (default 0.0); raising it trades sensitivity for specificity.
- `--pretreat` changes the response rule for patients who progressed on prior immunotherapy — not applicable here (ipilimumab was first-line).
- For a confidence interval on AUC: `ci.auc(TIDE_roc)`.

Extensions

- Cross-reference TIDE's `Dysfunction` and `Exclusion` scores with the CYT and CTLA4 correlations from Part 1 — a coherent story: high CTL → dysfunction axis, low CTL → exclusion axis.
- Compare TIDE's prediction with the `immunedeconv` CD8/T-cell estimates from Part 3.

7. Checklist, outputs, summary

Checklist

ID	Check	Result
ck1	Expression matrix loads, CPM col-Sums = 1e6	✓ 42 × 1e6

ID	Check	Result
ck2	Metadata reordered to exprsn	✓ `all(rownames(metadata)==colnames(exprsn))` TRUE
ck3	CYT median split	✓ 21 high / 21 low, no ties
ck4	KM plot + risk table render	✓
ck5	Cox model converges (6 covariates, 29 events)	✓ Concordance 0.679
ck6	Forest plot renders	✓
ck7	seven deconvolution methods complete	✓ (MCPcounter fixed — APP-01 correction applied)
ck8	quanTIseq rmgenes variant overrides first call	✓ 11 × 42
ck9	Stacked bar / dot / heatmap / row-scaled heatmap all render	✓
ck10	tidepy completes, output byte-size matches teacher's	✓ 10,259 B
ck11	res_tide reordered to metadata	✓
ck12	ROC + smoothed ROC render with AUC annotation	✓ 0.787 / 0.772
ck13	All outputs in `results/` by step, no raw data in repo	✓

Output manifest (31 files, no raw data)

```
02_gene_correlation/   GZMA_cor_PRF1.png, CTLA4_cor_CYT.png, scripts/part1_gene_correlation.R
03_survival_analysis/  survival_CYT.png, cox.png, scripts/part2_survival_analysis.R
04_immune_infiltration/ qts_barplot.png, qts_scatterplot.png, qts_heatmap.png,
                        qts_heatmap_row_scaled.png, CIBERSORT-Results.txt,
                        mcp_counter_blocked.txt, log_part3.txt,
                        deconv_methods/{res_mcp_counter,res_epic,res_quantiseq,
                        res_quantiseq_rmgenes,res_xcell,res_cibersort,
                        res_cibersort_abs,res_timer}.tsv,
                        scripts/part3_immune_infiltration.R,
                        scripts/part3_mcp_counter_fix.R
05_tide/               res_tide.txt, tide_heatmap.png, tide_predicted_responder.png,
                        tide_responder.png, roc.png, roc_smooth.png,
                        log_tidepy.txt, scripts/run_tidepy.sh, scripts/part4_tide_analysis.R
```

Summary

Lesson 3 is a pure downstream analysis — no FASTQ, no alignment, no SLURM. All four parts ran on the login node in R 4.3.3 (plus one `tidepy` shell call) in ~6 minutes. Seven verified pitfalls are logged; one (APP-01, MCPcounter) was a hard blocker during the as-written run (conda MCPcounter 1.1.0 hardcodes an **http** GitHub fetch for its marker genes on a login node where http-to-GitHub is reset) — the correction was subsequently executed and verified (local https-fetched markers passed as explicit arguments), so all seven deconvolution methods now have results. A duplicate-mapping finding was recorded: MCPcounter's 11 output rows include `Monocyte` and `Macrophage/Monocyte` with identical values because immunedeconv maps the single source population `Monocytic lineage` to both names (10 independent populations, not 11). The TIDE output is byte-size-identical to the teacher's pre-baked copy, and the ROC AUC (0.79) reproduces the published melanoma discrimination.

Appendix A. Deviation log (this run, condensed but verbatim-faithful)

- **Interactive → Rscript:** each part scripted as `partN_*.R`, run via `Rscript` under `LC_ALL=C`/`LANG=C` (RNA-seq R-segment segfault guard, project memory). Code content is verbatim from the tutorial; the wrapper is `setwd()` + diagnostic `cat()`/`print()` only.
- **Cross-part object sharing:** the tutorial runs in one interactive R session where `exprsn`, `sig`, `metadata`, `res_qts` flow across code blocks. Scripted as four processes, each script re-derives the objects it needs (re-reads the expression matrix, recomputes CYT). Code content unchanged; only the load is repeated.
- **Part 4 input path:** tutorial loads the teacher's pre-baked `/lustre1/share/rnaimmune/res_tide.txt`. The run instead loads its own `tidepy` output `results/05_tide/res_tide.txt` (byte-size-identical). Tutorial code block 28′s comment explicitly documents this as the intended usage when you run tidepy yourself.
- **MCPcounter (APP-01):** the lesson-3 run wrapped it in `tryCatch` and recorded the block; a subsequent correction (`scripts/part3_mcp_counter_fix.R`) ran it with local https-fetched markers and wrote `deconv_methods/res_mcp_counter.tsv`. The `mcp_counter_blocked.txt` file records both the original block and the applied fix.
- **Deconvolution results saved:** tutorial only `print()`s `res_qts`; the run additionally writes each method's matrix to `deconv_methods/*.tsv` (additive, non-destructive).
- **CIBERSORT-Results.txt:** a stray copy from a 3-sample probe run was removed from the repo root; the 42-sample production copy in `04_immune_infiltration/` is kept.

Appendix B. Reproduction

```
source /lustre1/share/miniconda3/etc/profile.d/conda.sh && conda activate rnaimmune
export LC_ALL=C; export LANG=C
cd /gpfs1/home/bjx131_pkuhpc/class/RNA-seq-application/results
Rscript 02_gene_correlation/scripts/part1_gene_correlation.R
Rscript 03_survival_analysis/scripts/part2_survival_analysis.R
Rscript 04_immune_infiltration/scripts/part3_immune_infiltration.R   # ~2 min, MCPcounter blocks
(APP-01)
bash    05_tide/scripts/run_tidepy.sh                                # ~11 s
Rscript 05_tide/scripts/part4_tide_analysis.R
# APP-01 correction (offline, needs ref/MCPcounter_*.txt):
Rscript 04_immune_infiltration/scripts/part3_mcp_counter_fix.R       # ~10 s
```

All outputs land in the respective step directories (scripts `setwd()` into them). MCPcounter is expected to fail with `Connection reset by peer` in the as-written run (logged in `mcp_counter_blocked.txt`); the fix script then runs it offline using the versioned `ref/MCPcounter_*.txt` marker files.